

DOCUMENTO TÉCNICO

Sistema de Finanzas Personales (FinanceApp)

Información General:

- **Nombre del Sistema:** FinanceApp
- **Curso:** Pruebas de Software
- **Entorno:** Python 3.12 (uv / pytest)

Integrantes:

- Arce Mayhua Leonardo Ruben
- Gallegos Condori Anette Isabel
- Jara Mamani Mariel Alisson
- Quispe Condori, Alvaro Raul

Índice

1. Descripción del Sistema	1
2. Validaciones de Integridad y Reglas de Negocio	2
3. Estrategia de Pruebas: Casos PE y AVL	2
3.1. Capa de Dominio (core/domain)	3
3.2. Capa de Aplicación (core/app)	4
3.3. Capa de Adaptadores de Salida (adapters/outbound)	5
3.4. Capa de Interfaz de Usuario / Adaptadores de Entrada (adapters/inbound)	6

1. Descripción del Sistema

El sistema **FinanceApp** es una solución para la gestión de finanzas personales, construida bajo el patrón de **Arquitectura Hexagonal**. Esta aproximación arquitectónica garantiza un desacoplamiento completo entre la lógica de negocio (el Núcleo de Dominio) y las tecnologías de infraestructura, interfaces de usuario y bases de datos.

La persistencia del sistema está desacoplada mediante puertos definidos en el núcleo y adaptadores de salida concretos (en este caso, repositorios en memoria para pruebas de caja negra). La entrada de datos y comandos se realiza a través de adaptadores de entrada (como una interfaz gráfica de usuario).

Las entidades clave que modelan las reglas de negocio son:

- **Account (Cuenta):** Representa un contenedor de fondos financieros con un balance y detalles bancarios.
- **Category (Categoría):** Clasifica los gastos e ingresos (ej. comida, transporte, salario).
- **Budget (Presupuesto):** Permite definir límites máximos de gastos para una categoría en un mes y año específicos.
- **Transaction (Transacción):** Registra el movimiento de dinero, sea un ingreso (sin categoría asociada) o un gasto (con categoría obligatoria).

El módulo de gestión de usuarios (User) ha sido omitido del informe final de integración debido a que la interfaz actual no requiere orquestar múltiples usuarios en la presente iteración.

2. Validaciones de Integridad y Reglas de Negocio

El sistema aplica validaciones estrictas en la inicialización de sus entidades y en la captura de entradas de usuario para evitar que se propague un estado inválido en la aplicación.

Entidad / Capa	Regla de Validación	Excepción / Error
Account	El nombre de la cuenta y el nombre del banco son obligatorios y no pueden estar vacíos ni consistir únicamente en espacios en blanco.	ValueError
Account	El depósito (ingreso) o retiro (gasto) en la cuenta debe ser estrictamente mayor que cero.	ValueError
Category	El nombre de la categoría es obligatorio y no puede estar vacío ni contener solo espacios en blanco.	ValueError
Budget	El monto límite debe ser estrictamente mayor a 0 (ej. mínimo 0.01).	ValueError
Budget	El mes del presupuesto debe ser un número entero en el rango de 1 a 12.	ValueError
Budget	El año del presupuesto debe ser igual o posterior a 1900.	ValueError
Transaction	El monto de la transacción debe ser estrictamente mayor a 0 (ej. mínimo 0.01).	ValueError
Transaction	Una transacción de tipo EXPENSE (gasto) requiere obligatoriamente estar asociada a una categoría existente.	ValueError
Transaction	Una transacción de tipo INCOME (ingreso) no permite asociar una categoría.	ValueError
Transaction	La descripción de la transacción es obligatoria y no puede estar vacía.	ValueError
Negocio (Account)	El monto de un gasto no puede superar el saldo actual disponible de la cuenta.	InsufficientFundsError
UI (Monto)	El monto ingresado en la interfaz debe ser convertible a decimal, no estar vacío y ser mayor que cero.	ValueError
UI (Nombre)	La entrada de texto para nombres de cuentas o categorías no puede estar vacía o llena de espacios.	ValueError
UI (Mes)	El mes provisto en la interfaz debe ser un entero convertible en el rango [1, 12].	ValueError
UI (Año)	El año provisto en la interfaz debe ser un entero convertible mayor o igual a 2000.	ValueError

3. Estrategia de Pruebas: Casos PE y AVL

Se ha implementado una suite completa de **74 pruebas unitarias** utilizando `pytest`. Estas pruebas aplican técnicas de Caja Negra de **Partición de Equivalencia (PE)** y **Análisis de Valores Límite (AVL)** en cada una de las capas de la arquitectura limpia.

A continuación se detallan de forma exhaustiva los casos de prueba implementados para cada capa del sistema.

3.1. Capa de Dominio (core/domain)

Esta capa contiene la lógica central del negocio e invariantes de datos de las entidades puras. Cuenta con **26 casos de prueba** (excluyendo el módulo de usuarios).

Caso de Prueba (Método)	Tipo	Escenario / Entrada	Resultado Esperado
test_valid_account	PE	Creación de cuenta con nombre («Savings») y banco («BCP») válidos.	Éxito. Saldo inicial a 0.00, estado activo.
test_register_income_and_expense_valid	PE	Registrar ingreso de 100.00 y luego un gasto de 40.00 en la cuenta.	Éxito. Saldo final resultante es de 60.00.
test_invalid_account_creation (Nombre vacío)	PE	Inicializar cuenta con nombre vacío («») y banco válido.	Falla. Lanza ValueError («Account name cannot be empty»).
test_invalid_account_creation (Nombre espacios)	PE	Inicializar cuenta con nombre con solo espacios (« »).	Falla. Lanza ValueError («Account name cannot be empty»).
test_invalid_account_creation (Banco vacío)	PE	Inicializar cuenta con nombre válido y banco vacío («»).	Falla. Lanza ValueError («Bank name cannot be empty»).
test_invalid_account_creation (Banco espacios)	PE	Inicializar cuenta con banco que contiene solo espacios (« »).	Falla. Lanza ValueError («Bank name cannot be empty»).
test_register_income_valid (0.01)	AVL	Registrar un ingreso por el valor mínimo positivo posible (0.01).	Éxito. Saldo final incrementado a 0.01.
test_register_income_valid (100)	PE	Registrar ingreso con monto positivo estándar (100.00).	Éxito. Saldo final incrementado a 100.00.
test_register_income_invalid (0.00)	AVL	Registrar ingreso con monto nulo (0.00) (límite no positivo).	Falla. Lanza ValueError («Income amount must be positive»).
test_register_income_invalid (-0.01)	AVL	Registrar ingreso con monto mínimo negativo (-0.01).	Falla. Lanza ValueError («Income amount must be positive»).
test_register_income_invalid (-100)	PE	Registrar ingreso con monto negativo estándar (-100.00).	Falla. Lanza ValueError («Income amount must be positive»).
test_register_expense_valid_boundary	AVL	Gasto por el monto exacto del saldo total disponible (100.00).	Éxito. Saldo resultante se reduce a 0.00.
test_register_expense_insufficient_funds (100.01)	AVL	Gasto de 100.01 en una cuenta con saldo disponible de 100.00.	Falla. Lanza InsufficientFundsError («Insufficient funds»).
test_register_expense_insufficient_funds (0.01)	AVL	Gasto de 0.01 en una cuenta con saldo disponible de 0.00.	Falla. Lanza InsufficientFundsError («Insufficient funds»).
test_valid_budget_boundaries (mínimos)	AVL	Presupuesto con límite 0.01, mes 1 (enero) y año 1900 (mínimo).	Éxito. Propiedades asignadas correctamente.

test_valid_budget_boundaries (límites)	AVL	Presupuesto con límite 100.00, mes 12 (diciembre) y año 2026.	Éxito. Propiedades asignadas correctamente.
test_invalid_budget_boundaries (límite 0)	AVL	Presupuesto con límite nulo (0.00).	Falla. Lanza ValueError («Budget limit must be greater than 0»).
test_invalid_budget_boundaries (límite negativo)	PE	Presupuesto con límite negativo (-1.00).	Falla. Lanza ValueError («Budget limit must be greater than 0»).
test_invalid_budget_boundaries (mes 0)	AVL	Presupuesto con mes inferior al límite (0).	Falla. Lanza ValueError («Month must be between 1 and 12»).
test_invalid_budget_boundaries (mes 13)	AVL	Presupuesto con mes superior al límite (13).	Falla. Lanza ValueError («Month must be between 1 and 12»).
test_invalid_budget_boundaries (año 1899)	AVL	Presupuesto con año anterior al límite (1899).	Falla. Lanza ValueError («Invalid year»).
test_valid_transaction_amount	AVL	Transacción con monto mínimo positivo aceptado (0.01).	Éxito. Monto asignado correctamente.
test_invalid_transaction_logic (gasto sin categoría)	PE	Crear transacción EXPENSE (gasto) con category_id = None.	Falla. Lanza ValueError («expense must have a category»).
test_invalid_transaction_logic (ingreso con categoría)	PE	Crear transacción INCOME (ingreso) con una categoría asociada.	Falla. Lanza ValueError («income must not have a category»).
test_invalid_transaction_amount (0.00)	AVL	Crear transacción con monto nulo (0.00).	Falla. Lanza ValueError («Transaction amount must be greater than 0»).
test_invalid_transaction_amount (-0.01)	AVL	Crear transacción con monto mínimo negativo (-0.01).	Falla. Lanza ValueError («Transaction amount must be greater than 0»).

3.2. Capa de Aplicación (core/app)

Esta capa contiene los servicios de aplicación (FinanceService) que orquestan los flujos de trabajo e implementan la lógica de negocio y persistencia abstracta. Cuenta con **11 casos de prueba**.

Caso de Prueba (Método)	Tipo	Escenario / Entrada	Resultado Esperado
test_create_account_persists_data	PE	Registrar cuenta y validar que es guardada en repositorio.	Éxito. Cuenta accesible vía repositorio.
test_calculate_budget_status_boundaries (Monto: 99.99)	AVL	Presupuesto de 100.00. Gasto total del mes = 99.99 (Límite - 0.01).	Éxito. Retorna exceeded = False y monto gastado 99.99.
test_calculate_budget_status_boundaries (Monto: 100.00)	AVL	Presupuesto de 100.00. Gasto total del mes = 100.00 (Límite exacto).	Éxito. Retorna exceeded = False y monto gastado 100.00.

test_calculate_budget_status_boundaries (Monto: 100.01)	AVL	Presupuesto de 100.00. Gasto total del mes = 100.01 (Límite + 0.01).	Éxito. Retorna exceeded = True y monto gastado 100.01.
test_register_income_updates_balance	PE	Registrar ingreso de 500.00 a través del servicio.	Éxito. Balance de cuenta se actualiza a 500.00 en repositorio.
test_register_expense_fails_insufficient_funds	PE	Intentar registrar un gasto de 10.00 en cuenta con balance 0.00.	Falla. Lanza InsufficientFundsError.
test_register_transaction_rejects_invalid_types (TRANSFER)	PE	Registrar transacción con tipo «TRANSFER» (no soportado).	Falla. Lanza ValueError («Invalid transaction type»).
test_register_transaction_rejects_invalid_types (LOAN)	PE	Registrar transacción con tipo «LOAN» (no soportado).	Falla. Lanza ValueError («Invalid transaction type»).
test_register_transaction_rejects_invalid_types (Vacío)	PE	Registrar transacción con tipo vacío («»).	Falla. Lanza ValueError («Invalid transaction type»).
test_list_active_accounts_filters_soft_deleted	PE	Desactivar una cuenta y luego listar las cuentas activas.	Éxito. Solo se lista la cuenta activa; la desactivada se excluye.
test_list_all_transactions_returns_full_history	PE	Registrar dos ingresos y listar el historial de transacciones.	Éxito. Se retornan las dos transacciones registradas.

3.3. Capa de Adaptadores de Salida (adapters/outbound)

Esta capa realiza las operaciones de persistencia de datos (repositorios en memoria). Cuenta con **12 casos de prueba**.

Caso de Prueba (Método)	Tipo	Escenario / Entrada	Resultado Esperado
test_add_and_get_active	PE	Añadir una cuenta activa y recuperarla por su ID.	Éxito. Se obtiene el objeto de cuenta correcto.
test_get_inactive_returns_none	PE	Buscar por ID una cuenta marcada como desactivada.	Retorna None (implementación de borrado lógico).
test_list_all_includes_inactive	PE	Llamar a list_all() con cuentas activas e inactivas en almacén.	Éxito. Retorna la lista con todas las cuentas.
test_list_by_category_and_period_boundaries (01-05-2026)	AVL	Buscar transacciones del mes 05-2026. Transacción el 01-05-2026 (primer día).	Éxito. La transacción es incluida en la lista.
test_list_by_category_and_period_boundaries (31-05-2026)	AVL	Buscar transacciones del mes 05-2026. Transacción el 31-05-2026 (último día).	Éxito. La transacción es incluida en la lista.
test_list_by_category_and_period_boundaries (30-04-2026)	AVL	Buscar transacciones del mes 05-2026. Transacción el 30-04-2026 (día previo).	Éxito. La transacción es excluida de la lista.

test_list_by_category_and_period_boundaries (01-06-2026)	AVL	Buscar transacciones del mes 05-2026. Transacción el 01-06-2026 (día posterior).	Éxito. La transacción es excluida de la lista.
test_list_by_category_and_period_boundaries (01-05-2025)	AVL	Buscar transacciones del mes 05-2026. Transacción el 01-05-2025 (mismo día/mes, año anterior).	Éxito. La transacción es excluida de la lista.
test_get_by_period_valid	PE	Recuperar presupuesto para una categoría y periodo existentes.	Éxito. Retorna el presupuesto correcto.
test_get_by_period_missing (Categoría errónea)	PE	Recuperar presupuesto buscando con categoría aleatoria inexistente.	Retorna None.
test_get_by_period_missing (Mes erróneo)	PE	Recuperar presupuesto buscando con mes diferente (4 en vez de 5).	Retorna None.
test_get_by_period_missing (Año erróneo)	PE	Recuperar presupuesto buscando con año diferente (2025 en vez de 2026).	Retorna None.

3.4. Capa de Interfaz de Usuario / Adaptadores de Entrada (adapters/inbound)

Esta capa valida la entrada de datos en la interfaz de usuario (cajas de texto) antes de ser enviadas a los servicios de dominio, y simula los comportamientos del sistema. Cuenta con **25 casos de prueba**.

Caso de Prueba (Método)	Tipo	Escenario / Entrada	Resultado Esperado
test_validate_amount_valid («150.50»)	PE	Validar cadena con decimal válido.	Éxito. Retorna Decimal («150.50»).
test_validate_amount_valid («200»)	PE	Validar cadena con número entero válido.	Éxito. Retorna Decimal («200»).
test_validate_amount_valid («0.01»)	AVL	Validar monto con valor límite inferior permitido («0.01»).	Éxito. Retorna Decimal («0.01»).
test_validate_amount_invalid («abc»)	PE	Validar entrada de caracteres no numéricos.	Falla. Lanza ValueError («Debe ingresar un número válido»).
test_validate_amount_invalid («»)	PE	Validar entrada vacía.	Falla. Lanza ValueError («El campo monto no puede estar vacío»).
test_validate_amount_invalid (« »)	PE	Validar entrada con solo espacios en blanco.	Falla. Lanza ValueError («El campo monto no puede estar vacío»).
test_validate_amount_invalid («-10»)	PE	Validar entrada con monto negativo.	Falla. Lanza ValueError («El monto debe ser mayor a 0»).

test_validate_amount_invalid («0»)	AVL	Validar entrada con monto nulo («0») (límite no permitido).	Falla. Lanza ValueError («El monto debe ser mayor a 0»).
test_validate_name_valid («Ahorros »)	PE	Validar nombre con espacios en los extremos.	Éxito. Retorna «Ahorros» (espacios eliminados con strip).
test_validate_name_valid («A»)	AVL	Validar nombre con longitud mínima de un carácter («A»).	Éxito. Retorna «A».
test_validate_name_invalid («»)	PE	Validar entrada de nombre vacía.	Falla. Lanza ValueError («El nombre no puede estar vacío»).
test_validate_name_invalid (« »)	PE	Validar entrada de nombre con solo espacios.	Falla. Lanza ValueError («El nombre no puede estar vacío»).
test_validate_month_valid («1»)	AVL	Validar mes en el límite inferior («1»).	Éxito. Retorna 1.
test_validate_month_valid («12»)	AVL	Validar mes en el límite superior («12»).	Éxito. Retorna 12.
test_validate_month_valid («6»)	PE	Validar mes intermedio («6»).	Éxito. Retorna 6.
test_validate_month_invalid («0»)	AVL	Validar mes por debajo del límite («0»).	Falla. Lanza ValueError («El mes debe estar entre 1 y 12»).
test_validate_month_invalid («13»)	AVL	Validar mes por encima del límite («13»).	Falla. Lanza ValueError («El mes debe estar entre 1 y 12»).
test_validate_month_invalid («marzo»)	PE	Validar entrada de mes no numérica.	Falla. Lanza ValueError («El mes debe ser un número entero»).
test_validate_year_valid («2000»)	AVL	Validar año en el límite inferior permitido («2000»).	Éxito. Retorna 2000.
test_validate_year_valid («2026»)	PE	Validar año normal en el futuro («2026»).	Éxito. Retorna 2026.
test_validate_year_invalid («1999»)	AVL	Validar año por debajo del límite («1999»).	Falla. Lanza ValueError («El año debe ser 2000 o posterior»).
test_validate_year_invalid («veinte»)	PE	Validar entrada de año no numérica.	Falla. Lanza ValueError («El año debe ser un número entero»).
test_register_income_updates_simulated_balance	PE	Registrar ingreso de 100 en DummyService.	Éxito. Balance simulado aumenta a 1100.00.

test_register_expense_fails_insufficient_funds	PE	Registrar gasto de 1000.01 en DummyService (superando balance inicial).	Falla. Lanza Insufficient-FundsError.
test_list_active_filters_deactivated	PE	Desactivar una cuenta en DummyService y listar activas.	Éxito. Retorna lista vacía (cuenta filtrada).